



Todo App

Complete CI/CD Setup Guide

Node.js + Express + MongoDB + Jenkins + Docker

Node.js 20

Express 5

MongoDB

Jest

Jenkins

Docker

Generated: July 1, 2026



Table of Contents

1. Overview — What We Built

1.1 Project Stack

1.2 Jenkins Pipeline Flow

2. Prerequisites

2.1 Software You Need to Install

2.2 Accounts You Need

3. Push to GitHub

3.1 Initialize Git

3.2 Create GitHub Repository

3.3 Push the Code

4. Install & Configure Docker

4.1 Installing Docker Desktop

4.2 Building the Docker Image

4.3 Running the App in Docker

4.4 Docker Hub Account & Push

5. Install & Configure Jenkins

5.1 Installing Jenkins

5.2 Installing Plugins

5.3 Configuring Credentials

5.4 Creating the Pipeline Job

5.5 Running the Pipeline

6. Understanding the Jenkins Pipeline

6.1 Stage-by-Stage Breakdown

6.2 Failure Scenarios

7. Running the App Locally

7.1 Without Docker

7.2 With Docker

8. Running Tests

9. Project File Structure

10. Troubleshooting

1. Overview — What We Built

This is a **Todo Application** built with Node.js, Express, and MongoDB. It includes a full **CI/CD pipeline** using Jenkins, with automated testing, code quality checks, security audits, Docker image building, and deployment readiness.

1.1 Project Stack

Component	Technology
Backend Framework	Express 5 (Node.js)
Database	MongoDB (via Mongoose ODM)
Frontend	Vanilla JavaScript + CSS (SPA)
Testing	Jest + Supertest + mongodb-memory-server
Code Quality	ESLint
CI/CD	Jenkins (Declarative Pipeline)
Containerization	Docker (multi-stage, production)

1.2 Jenkins Pipeline Flow

Pipeline Sequence:

- Checkout** — Pull code from Git
- Install Dependencies** — npm ci
- Code Quality Checks (Parallel)** — Lint + Security Audit + Unit Tests all run concurrently
- Build Check** — Syntax validation
- Docker Build** — Build production image (main branch only)
- Docker Push** — Push to registry (main branch only)
- Post** — Cleanup + notifications

2. Prerequisites

2.1 Software You Need to Install

☐ **Git** — git-scm.com

☐ **Node.js 20+** — nodejs.org (includes npm)

☐ **Docker Desktop** — docker.com

☐ **Jenkins** — jenkins.io (WAR or installer)

☐ **MongoDB** (for local testing) — mongodb.com

☐ **MongoDB Compass** (optional, GUI) — mongodb.com/products/compass

Note: You don't need Docker running to develop locally. Docker is only needed when you want to build and run the containerized app or use the Jenkins pipeline's Docker build/push stages.

2.2 Accounts You Need

☐ **GitHub (or GitLab/Bitbucket)** — to host your repository

☐ **Docker Hub** — hub.docker.com (free account) to store Docker images

3. Push to GitHub

3.1 Initialize Git

If Git is not already initialized in the project:

```
cd C:\Users\arunk\monogdb
git init
git add .
git commit -m "Initial commit - Todo App with full CI/CD pipeline"
```

3.2 Create a GitHub Repository

1. Go to github.com/new
2. Repository name: **todo-app** (or any name you prefer)
3. Keep it **Public** (or Private — either works)
4. Do **not** initialize with README, .gitignore, or license (we already have these)
5. Click "**Create repository**"

3.3 Push the Code

```
# Replace YOUR_USERNAME with your GitHub username
git remote add origin https://github.com/YOUR_USERNAME/todo-app.git
git branch -M main
git push -u origin main
```

✅ **Done!** Your code is now on GitHub. Jenkins will pull from this repository.

4. Install & Configure Docker

⚠ **Do you need Docker?** The app runs fine without Docker for local development. Docker is only needed for:

- Building a production container image
- The Jenkins pipeline's Docker build/push stages
- Running the app in a containerized environment

4.1 Installing Docker Desktop

1. Download Docker Desktop from docker.com
2. Run the installer and follow the setup wizard
3. After installation, **restart your computer**
4. Open Docker Desktop — it should show the Docker whale icon
5. Verify it works:

```
docker --version
docker info
```

4.2 Building the Docker Image

Navigate to the project folder and build:

```
cd C:\Users\arunk\monogdb
docker build -t todo-app .
```

This uses the Dockerfile in the project to create a production-ready image with:

- Node.js 20 (Alpine Linux — very small footprint)
- Only production dependencies (no dev dependencies like Jest)
- Non-root user for security
- Health check configured

4.3 Running the App in Docker

```
# Run the container (replace MONGO_URI with your MongoDB connection)
docker run -p 3000:3000 -e MONGO_URI="mongodb://host.docker.internal:27017/todo-app"
todo-app
```

Then open <http://localhost:3000> in your browser.

 **Note:** `host.docker.internal` is a special Docker DNS name that lets the container reach MongoDB running on your host machine. If MongoDB is also running in Docker, use the container name instead.

4.4 Docker Hub Account & Push

1. Create a free account at hub.docker.com
2. Log in from command line:

```
docker login -u YOUR_DOCKER_HUB_USERNAME
```

Then tag and push your image:

```
# Tag with your Docker Hub username
docker tag todo-app YOUR_USERNAME/todo-app:latest
docker tag todo-app YOUR_USERNAME/todo-app:v1.0.0

# Push to Docker Hub
docker push YOUR_USERNAME/todo-app:latest
docker push YOUR_USERNAME/todo-app:v1.0.0
```

 **Done!** Your image is now on Docker Hub. The Jenkins pipeline automates this for you on every push to `main`.

5. Install & Configure Jenkins

5.1 Installing Jenkins

Option A: Using the Windows installer

1. Download the Windows installer from jenkins.io/download
2. Run the installer — it will install Jenkins as a Windows service
3. Complete the setup wizard at <http://localhost:8080>
4. Use the initial admin password from:
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
5. Install the **"suggested plugins"**
6. Create your admin user

Option B: Using Docker (easier)

```
docker run -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts
```

 **Recommended:** The Docker approach is simpler and includes Docker CLI inside Jenkins automatically (with the socket mount). Access Jenkins at <http://localhost:8080>.

5.2 Required Jenkins Plugins

If you installed "suggested plugins", most are already there. Verify these are installed:

Plugin	Purpose
Pipeline	For Declarative Pipeline syntax
Git	To pull code from GitHub
Docker Pipeline	To build/push Docker images

Credentials Binding	To securely use credentials
Blue Ocean	Better pipeline UI (optional but nice)

To install plugins: **Manage Jenkins** → **Plugins** → **Available plugins**

5.3 Configuring Credentials

Go to **Manage Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**

Credential 1: Docker Hub (for Docker push)

Field	Value
Kind	Username with password
Username	<i>Your Docker Hub username</i>
Password	<i>Your Docker Hub password or access token</i>
ID	docker-registry-credentials
Description	Docker Hub credentials

Credential 2: MongoDB URI (for pipeline tests)

Field	Value
Kind	Secret text
Secret	mongodb://localhost:27017/todo-app
ID	mongo-uri
Description	MongoDB connection string

5.4 Creating the Pipeline Job

1. From Jenkins dashboard, click **"New Item"**
2. Enter name: **todo-app-pipeline**
3. Select **"Pipeline"** and click OK

4. Scroll to the **"Pipeline"** section
5. Set **Definition** → **"Pipeline script from SCM"**
6. Set **SCM** → **Git**
7. Enter Repository URL: `https://github.com/YOUR_USERNAME/todo-app.git`
8. Branch: `*/main`
9. Script Path: `Jenkinsfile`
10. Click **"Save"**

 **Jenkins will automatically find the Jenkinsfile** from your repository and run the pipeline!

5.5 Running the Pipeline

1. From the pipeline page, click **"Build Now"**
2. Watch the pipeline run stage by stage in the **"Stage View"**
3. You'll see parallel stages (Lint, Security, Tests) running concurrently
4. If on main branch, Docker build and push will run after all checks pass

 **Pro tip:** Install the **Blue Ocean** plugin for a much nicer pipeline visualization. Click **"Open Blue Ocean"** from the Jenkins dashboard.

6. Understanding the Jenkins Pipeline

6.1 Stage-by-Stage Breakdown

#	Stage	What it does	What can go wrong
1	Checkout	Pulls code from GitHub	Network down, bad credentials, wrong branch
2	Install Dependencies	Runs <code>npm ci</code> to install packages	Registry unreachable, lockfile mismatch, disk full
3a	Lint (parallel)	ESLint code quality check	Code style violations exceed threshold
3b	Security Audit (parallel)	<code>npm audit</code> checks for CVEs	High/critical vulnerabilities found
3c	Unit Tests (parallel)	Runs 31 Jest tests with in-memory MongoDB	Tests fail, flaky tests, timeout
4	Build Check	Syntax validation of all JS files	Syntax errors in source code
5	Docker Build (main only)	Builds production Docker image	Docker not available, Dockerfile error
6	Docker Push (main only)	Pushes image to Docker Hub	Registry credentials expired, network error

6.2 Failure Scenarios

✖ Pipeline fails at Docker Build/Push

Fix: Ensure Docker is installed on the Jenkins node and the `docker-registry-credentials` credential is configured correctly in Jenkins.

✖ Tests fail

Fix: Run `npm test` locally to reproduce. Check test output in the Jenkins console log.

✖ Lint fails

Fix: Run `npm run lint` locally and fix the issues. Or adjust the ESLint config if rules are too strict.

✖ Security audit fails

Fix: Run `npm audit` to see the vulnerabilities. Update affected packages with `npm update` OR `npm audit fix`.

7. Running the App Locally

7.1 Without Docker

1. Make sure MongoDB is running locally (default: `mongodb://localhost:27017`)
2. Install dependencies: `npm install`
3. Start the app: `npm start`
4. Open <http://localhost:3000>

```
# Quick start
npm install
npm start
# App runs at http://localhost:3000
```

7.2 With Docker

```
# Build the image
docker build -t todo-app .

# Run with MongoDB on host
docker run -p 3000:3000 -e MONGO_URI="mongodb://host.docker.internal:27017/todo-app"
todo-app
```

 **Tip:** If you don't have MongoDB installed, use Docker Compose:

```
# docker-compose.yml
version: '3'
services:
  mongodb:
    image: mongo:7
    ports:
      - "27017:27017"
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - MONGO_URI=mongodb://mongodb:27017/todo-app
```

```
depends_on:  
  - mongodb
```

8. Running Tests

The project includes **31 comprehensive unit tests** covering all API routes:

API Route	Tests	Edge Cases
POST /api/todos	5	valid, empty, missing, trimmed, >200 chars
GET /api/todos	3	empty list, sort order, sub-items
GET /api/todos/:id	3	found, 404, invalid ObjectId
PUT /api/todos/:id	6	update title, toggle, both, empty, 404
DELETE /api/todos/:id	2	existing, non-existent
POST /:id/items	4	add, multiple, empty, 404
PUT /:id/items/:itemId	5	title, toggle, 404 todo, 404 sub-item
DELETE /:id/items/:itemId	3	delete, 404 todo, 404 sub-item
Full workflow	1	create → sub-items → toggle → delete

Run tests:

```
# Run all tests
npm test

# Run in watch mode
npm run test:watch

# Run syntax check only
npm run test:syntax
```

 **How it works:** Tests use `mongodb-memory-server` which downloads and runs a real MongoDB binary in-memory. No separate MongoDB installation is needed for testing. Each test run gets a completely fresh database.

9. Project File Structure

```
C:\Users\arunk\monogdb\
├── Jenkinsfile           # CI/CD pipeline definition
├── Dockerfile            # Production container build
├── package.json          # Dependencies & scripts
├── server.js             # Express app entry point
├── .eslintrc.json        # ESLint configuration
├── .gitignore            # Files to exclude from Git
├──
├── routes/
│   └── todos.js          # All API route handlers
├──
├── models/
│   └── Todo.js           # Mongoose schema (todos + sub-items)
├──
├── public/
│   ├── index.html        # SPA frontend
│   ├── style.css         # Styling
│   └── script.js         # Frontend logic
├──
├── __tests__/
│   ├── setup.js          # MongoDB memory server helper
│   └── todos.test.js     # 31 comprehensive API tests
├──
├── scripts/
│   └── generate-guide.js  # This PDF generator
```

10. Troubleshooting

Problem	Solution
npm install fails	Delete <code>node_modules</code> and <code>package-lock.json</code> , then run <code>npm install</code> again
Docker build fails	Make sure Docker Desktop is running. Check the Dockerfile for syntax errors
Jenkins can't find Git repo	Verify the repository URL in the pipeline config. Make sure the repo is not private without SSH keys configured
Tests fail with MongoDB timeout	<code>mongodb-memory-server</code> needs to download the MongoDB binary on first run. This can take a few minutes. Increase <code>testTimeout</code> in <code>package.json</code>
Docker push fails in Jenkins	Verify <code>docker-registry-credentials</code> exists in Jenkins credentials. Check that the Docker Hub username/password is correct
Port 3000 already in use	Change the port: <code>set PORT=3001 && npm start</code> (Windows) or <code>PORT=3001 npm start</code> (Mac/Linux)
MongoDB connection refused	Make sure MongoDB is running. Default: <code>mongodb://localhost:27017</code> . Check if the MongoDB service is started
ESLint not found in pipeline	The lint stage is conditional — it only runs if an ESLint config file exists. We have one, so verify <code>.eslintrc.json</code> is committed

You're all set!

Your Todo App now has a complete CI/CD pipeline with:

-  Automated testing (31 tests)
-  Code quality checks (ESLint)
-  Security auditing (npm audit)
-  Parallel pipeline stages for faster feedback
-  Docker containerization
-  Automated Docker build & push
-  Jenkins Declarative Pipeline